

Stack Overflow is a community of 4.7 million programmers, just like you, helping each other. Join them; it only takes a minute:

Sign up



What is the difference between partitioning and bucketing a table in Hive ?



I know both is performed on a column in the table but how is each operation different.

hadoop hive

asked Oct 2 '13 at 2:09



NishM

125 1 1 9

5 Answers

Partitioning data is often used for distributing load horizontally, this has performance benefit, and helps in organizing data in a logical fashion. Example like if we are dealing with large *employee* table and often run queries with WHERE clauses that restrict the results to a particular country or department . For a faster query response Hive table can be PARTITIONED BY (country STRING, DEPT STRING), Partitioning tables changes how Hive structures the data storage and Hive will now create subdirectories reflecting the partitioning structure like .
.../employees/country=ABC/DEPT=XYZ. If query limits for employee from country ABC t will only scan the contents of one directory ABC. This can dramatically improve query performance, but only if the partitioning scheme reflects common filtering. Partitioning feature is very useful in Hive, however, a design that creates too many partitions may optimize some queries, but be detrimental for other important queries. Other drawback is having too many partitions is the large number of Hadoop files and directories that are created unnecessarily and overhead to NameNode since it must keep all metadata for the file system in memory.

Bucketing is another technique for decomposing data sets into more manageable parts. For example, suppose a table using the date as the top-level partition and the employee_id as the second-level partition leads to too many small partitions. Instead, if we bucket the employee table and use employee_id as the bucketing column, the value of this column will be hashed by a user-defined number into buckets. Records with the same employee_id will always be stored in the same bucket. Assuming the number of employee_id is much greater than the number of buckets, each bucket will have many employee_id. While creating table you can specify like CLUSTERED BY (employee_id) INTO XX BUCKETS ; where XX is the number of buckets . Bucketing has several advantages. The number of buckets is fixed so it does not fluctuate with data. If two tables are bucketed by employee_id, Hive can create a logically correct sampling. Bucketing also aids in doing efficient map-side joins etc.

answered Oct 2 '13 at 6:37



Navneet Kumar

995 7 19

Very helpful. Thank you – NishM Oct 7 '13 at 3:12

Thanks Navneet. However, can you elaborate how bucketing happens with partitioning ? Suppose if we specify 32 buckets in the CLUSED BY clause and the CREATE TABLE statement also contains the Partitioning clause, how partitions and buckets will be managed together ? Does number of partitions will be limited to 32 ? OR for each partition, 32 buckets will be created ? Is every bucket an HDFS file ? – sgsi Feb 2 at 1:08

1 A hive table can have both partitioning and bucketing. Based on your partition clause , for each partition will have 32 buckets created. Yes HDFS file. – Navneet Kumar Feb 2 at 16:57

Thanks Navneet. – sgsi Feb 3 at 0:36

I think I am late in answering this question, but it keep coming up in my feed.

Navneet has provided excellent answer. Adding to it visually.

Partitioning helps in elimination of data, if used in WHERE clause, where as bucketing helps in organizing data in each partition into multiple files, so as same set of data is always written in same bucket. Helps a lot in joining of columns.

Suppose, you have a table with five columns, name, server_date, some_col3, some_col4 and some_col5. Suppose, you have partitioned the table on *server_date* and bucketed on *name* column in 10 buckets, your file structure will look something like below.

1. server_date=xyz
 - 00000_0
 - 00001_0
 - 00002_0
 -
 - 00010_0

Here *server_date*=xyz is the partition and *000* files are the buckets in each partition. Buckets are calculated based on some hash functions, so rows with *name*=Sandy will always go in same bucket.

answered Sep 14 at 12:19



Priyesh

111 8

Hive Partitioning:

Partition divides large amount of data into multiple slices based on value of a table column(s).

Assume that you are storing information of people in entire world spread across 196+ countries spanning around 500 crores of entries. If you want to query people from a particular country (Vatican city), in absence of partitioning, you have to scan all 500 crores of entries even to fetch one million rows for a particular country. If you partition the table based on country, you can fine tune querying process by just checking the data for only one country partition. Hive partition creates a separate directory for a column(s) value.

Pros:

1. Distribute execution load horizontally
2. Faster execution of queries in case of partition with low volume of data. e.g. Get the population from "**Vatican city**" returns very fast instead of searching entire population of world.

Cons:

1. Possibility of too many small partition creations - too many directories.
2. Effective for low volume data for a given partition. But some queries like group by on high volume of data still take long time to execute. e.g. Grouping of population of China will take long time compared to grouping of population in Vatican city. Partition is not solving responsiveness problem in case of data skewing towards a particular partition value.

Hive Bucketing:

Bucketing decomposed data into more manageable or equal parts.

With partitioning, there is a possibility that you can create multiple small partitions based on column values. If you for bucketing, you are just restricting number of buckets to store the data. This number is defined during table creation scripts.

Pros

1. Due to equal volumes of data in each partition, joins at Map side will be quicker.
2. Faster query response like partitioning

Cons

1. You can define number of buckets during table creation but loading of equal volume of data to be done manually by programmers.

answered Nov 11 at 7:01



ravindra

3,209 2 4 25

There are a few details missing from the previous explanations. To better understand how partitioning and bucketing works, you should look at how data is stored in hive. Let's say you have a table

```
CREATE TABLE mytable (
  name string,
  city string,
  employee_id int )
PARTITIONED BY (year STRING, month STRING, day STRING)
```

```
CLUSTERED BY (employee_id) INTO 256 BUCKETS
```

then hive will store data in a directory hierarchy like

```
/user/hive/warehouse/mytable/y=2015/m=12/d=02
```

So, you have to be careful when partitioning, because if you for instance partition by `employee_id` and you have millions of employees, you'll end up having millions of directories in your file system. The term '**cardinality**' refers to the number of possible value a field can have. For instance, if you have a 'country' field, the countries in the world are about 300, so cardinality would be ~300. For a field like 'timestamp_ms', which changes every millisecond, cardinality can be billions. In general, when choosing a field for partitioning, it should not have a high cardinality, because you'll end up with way too many directories in your file system.

Clustering aka bucketing on the other hand, will result with a fixed number of files, since you do specify the number of buckets. What hive will do is to take the field, calculate a hash and assign a record to that bucket. But what happens if you use let's say 256 buckets and the field you're bucketing on has a low cardinality (for instance, it's a US state, so can be only 50 different values) ? You'll have 50 buckets with data, and 206 buckets with no data.

Someone already mentioned how partitions can dramatically cut the amount of data you're querying. So in my example table, if you want to query only from a certain date forward, the partitioning by year/month/day is going to dramatically cut the amount of IO. I think that somebody also mentioned how bucketing can speed up joins with other tables *that have exactly the same bucketing*, so in my example, if you're joining two tables on the same `employee_id`, hive can do the join bucket by bucket (even better if they're already sorted by `employee_id` since it's going to to a mergesort which works in linear time).

So, bucketing works well when the field has high cardinality and data is evenly distributed among buckets. Partitioning works best when the cardinality of the partitioning field is not too high.

Also, *you can partition on multiple fields*, with an order (year/month/day is a good example), while *you can bucket on only one field*.

answered Dec 6 at 23:42



Roberto Congiu

480 2 7

Partitions: Partitions are fundamentally horizontal slices of data which allow larges sets of data to be segmented into more manageable chunks.

```
CREATE TABLE customer (  
  id          INT,  
  name        STRING,  
  address1    STRING )PARTITION BY (REGION STRING, country STRING );
```

Multiple 'slicer' or partitioning columns are supported (i.e. Region/Country).

For The difference between partitioning and bucketing a table in Hive refer [this link](#)

answered Nov 3 at 12:18



Sayyed M

30 7